Graduate Systems (CSE638)

PA02: Cost of Data Movement in Network I/O

Piyush Kandwal
Roll Number: MT25076

6 February 2026

Contents

1 Introduction **3**

2 Part A: Multithreaded Socket Implementations **3**

2.1 Common Design 3

2.2 A1: Two-Copy Implementation 3

2.3 A2: One-Copy Implementation 4

2.4 A3: Zero-Copy Implementation 4

3 Part B: Profiling and Measurement Methodology **4**

3.1 Metrics Collected 4

3.2 Experimental Setup 4

4 Part C: Experimental Results and Analysis **5**

4.1 Latency Analysis 5

4.2 Throughput Analysis 6

5 Part D: Cache and Scaling Analysis **7**

5.1 Cache Miss Behavior 7

5.2 CPU Cycles per Byte 8

5.3 Effect of Thread Count 8

6 Key Observations **8**

6.1 Unexpected Result 9

7 AI Tool Usage Declaration **9**

7.1 Components Where AI Assistance Was Used 9

7.1.1 Code Development and Implementation 9

7.1.2 Experimental Design and Parameter Selection 10

7.1.3 Running Experiments and Data Collection 10

7.1.4 Plot Generation and Visualization 10

7.1.5 Data Interpretation and Report Writing 11

7.2 Summary and Responsibility Statement 11

8 Conclusion **11**

9 GitHub Repository **11**

1 Introduction

Modern high-performance network applications are often limited not by raw network bandwidth, but by the overhead of data movement between user space, kernel space, and the network interface. Traditional socket-based communication involves multiple memory copies, which increase CPU utilization and cache pressure.

This assignment experimentally studies the cost of data movement in network I/O by implementing and comparing three TCP-based communication mechanisms:

- **Two-Copy Communication**: Using `send()` / `recv()`
- **One-Copy Communication**: Using `sendmsg()` with scatter–gather I/O
- **Zero-Copy Communication**: Using `sendmsg()` with `MSG_ZEROCOPY`

The implementations are evaluated using micro-architectural metrics such as CPU cycles, cache misses, throughput, latency, and scalability across varying message sizes and thread counts.

2 Part A: Multithreaded Socket Implementations

2.1 Common Design

All implementations follow a TCP client–server architecture. The server accepts multiple concurrent client connections and creates one thread per client. Each server thread repeatedly sends a fixed-size message to its client.

The message is defined as a structure containing eight dynamically allocated string fields, ensuring that all message data is heap-allocated. Message size and thread count are parameterized at runtime. The client receives data continuously for a fixed duration.

2.2 A1: Two-Copy Implementation

The baseline implementation uses `send()` and `recv()`. Since `send()` requires a single contiguous buffer, the eight message fields are first serialized into a temporary buffer in user space.

This results in two primary data copies:

- User-space buffer to kernel socket buffer
- Kernel socket buffer to user space on the receiver

In practice, additional internal kernel copies may occur due to protocol processing.

2.3 A2: One-Copy Implementation

The one-copy implementation uses `sendmsg()` with an array of `iovec` structures, each pointing directly to one message field. This eliminates the explicit user-space serialization copy. However, the kernel still performs one copy into socket buffers.

2.4 A3: Zero-Copy Implementation

The zero-copy implementation uses `sendmsg()` with the `MSG_ZEROCOPY` flag. The kernel pins user-space pages and transfers data directly via DMA, avoiding data copies in the steady state. Completion notifications are retrieved from the socket error queue.

3 Part B: Profiling and Measurement Methodology

3.1 Metrics Collected

The following metrics were collected:

- Throughput (Gbps)
- Application-level latency (μs)
- CPU cycles
- Cache misses (L1 and LLC)
- Context switches

Micro-architectural metrics were collected on the server using `perf stat`, while application-level metrics were measured at the client.

3.2 Experimental Setup

All experiments were executed using Linux network namespaces connected via a `veth` pair. Measurements were taken across four message sizes (1K, 4K, 16K, 64K) and four thread counts (1, 2, 4, 8).

An automated bash script was used to compile all implementations, execute experiments, collect profiling data, and store results in CSV format.

4 Part C: Experimental Results and Analysis

4.1 Latency Analysis

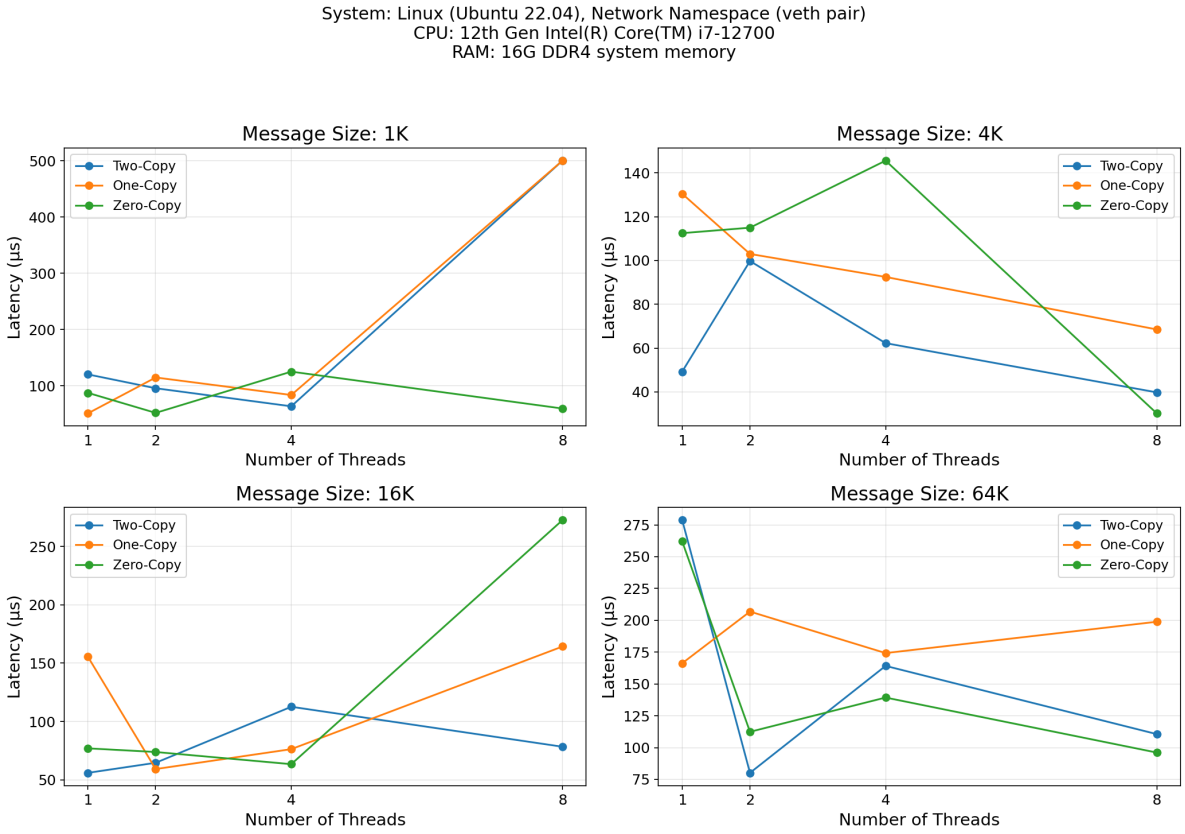


Figure 1: Application-level latency vs thread count for different message sizes. Latency generally increases with thread count due to scheduling overhead, cache contention, and socket buffer pressure.

4.2 Throughput Analysis

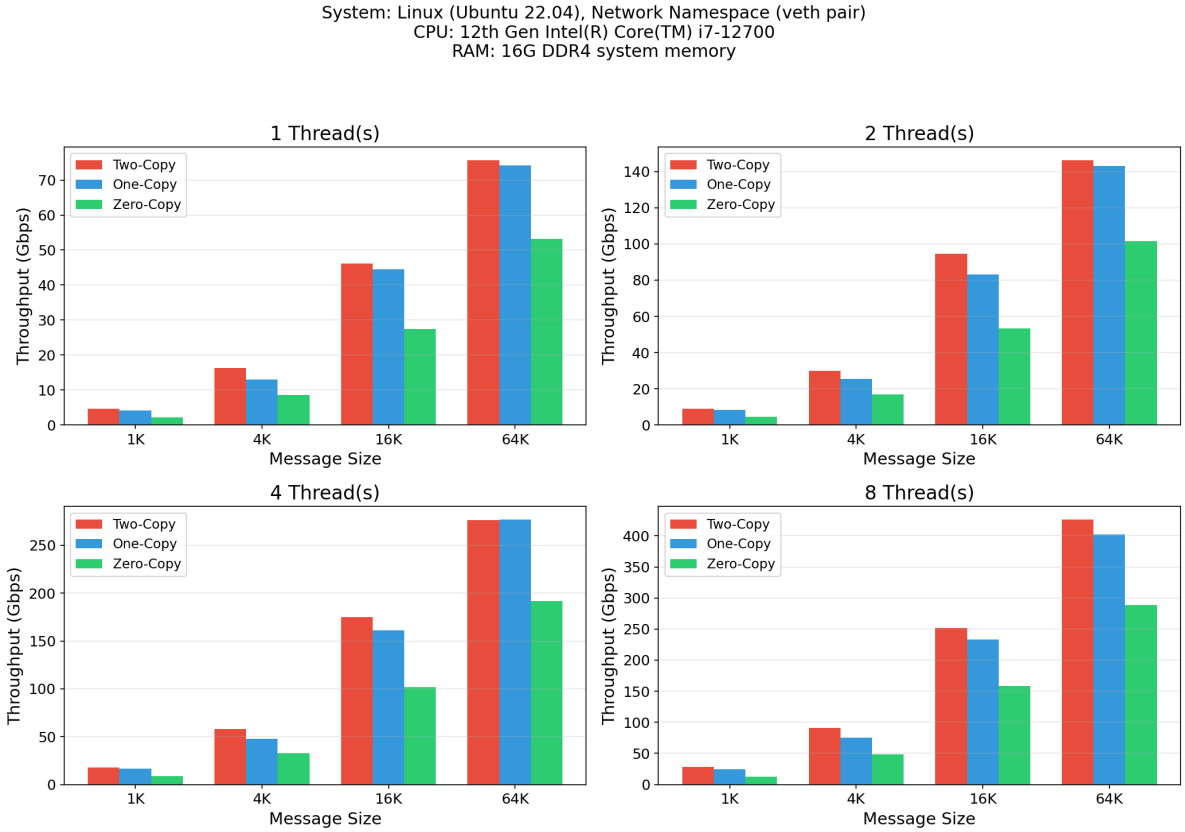


Figure 2: Throughput vs message size for varying thread counts. Throughput increases with message size as fixed overheads are amortized. One-copy outperforms two-copy at medium sizes, while zero-copy benefits appear primarily for large messages.

5 Part D: Cache and Scaling Analysis

5.1 Cache Miss Behavior

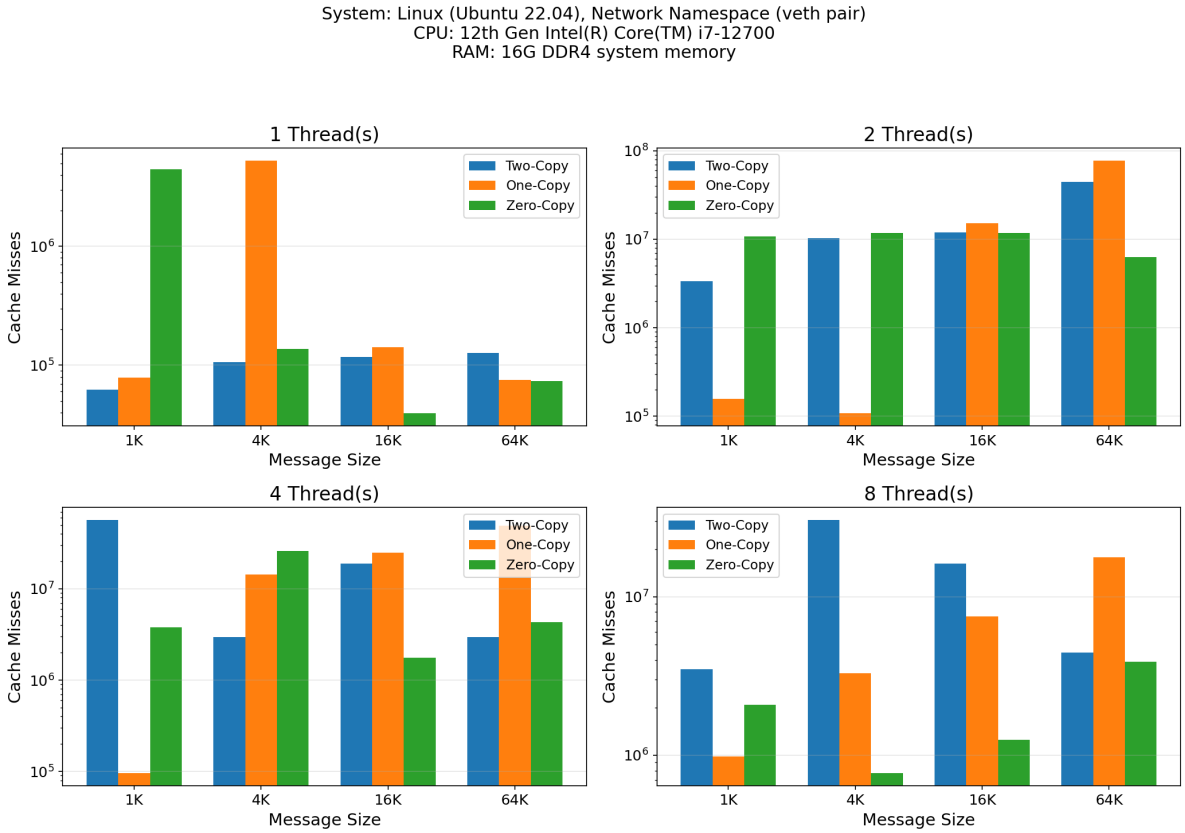


Figure 3: Cache misses vs message size for different thread counts. Two-copy communication incurs the highest cache miss rates due to repeated memory copying. One-copy reduces cache pressure, while zero-copy shows higher misses for small messages but fewer misses for large messages where DMA bypasses CPU caches.

5.2 CPU Cycles per Byte

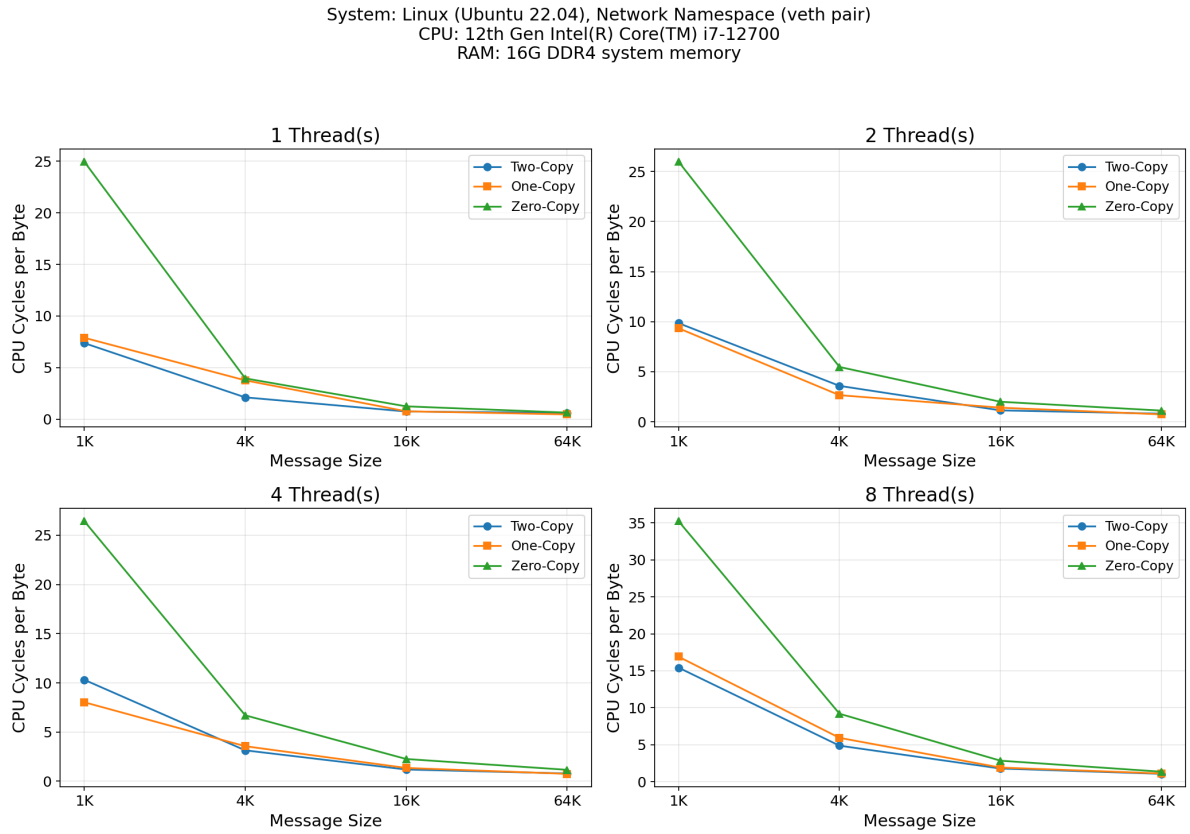


Figure 4: CPU cycles per byte transferred. Cycles per byte decrease as message size increases due to amortization of system call and protocol overheads. Zero-copy has higher overhead for small messages but converges to the lowest cost for large transfers.

5.3 Effect of Thread Count

Increasing thread count amplifies cache contention and context switching overhead in copy-based implementations. Zero-copy scales better for large messages due to reduced cache line traffic, although page pinning overhead limits benefits at very high concurrency.

6 Key Observations

1. Zero-copy does not always yield the best throughput due to high setup and page pinning overheads for small messages.

2. LLC cache misses show the greatest reduction since eliminating copies reduces cache line pollution.

3. Increasing thread count increases cache contention and scheduling overhead.

4. One-copy outperforms two-copy at medium message sizes where serialization overhead becomes significant.

5. Zero-copy outperforms two-copy only at large message sizes where fixed overheads are amortized.

6.1 Unexpected Result

An unexpected observation was that zero-copy exhibited higher cache misses for small message sizes. This is explained by page pinning, metadata management, and DMA setup overhead dominating the execution, outweighing the benefit of avoided data copies.

7 AI Tool Usage Declaration

As required by the course policy, this section provides a detailed and explicit declaration of all components of this assignment where Large Language Model (LLM) tools were used, including the nature of usage and representative prompts. AI tools were extensively used throughout the assignment; however, all outputs were critically reviewed, modified, tested, and validated by me before final submission. I take full responsibility for the correctness of the code, results, and analysis.

7.1 Components Where AI Assistance Was Used

7.1.1 Code Development and Implementation

AI tools were used during the development of the C client–server programs to:

- Generate initial code skeletons for socket programming
- Suggest correct usage patterns for `send()`, `sendmsg()`, `iovec`, and `MSG_ZEROCOPY`
- Help refactor code to meet assignment constraints
- Debug compilation errors, runtime crashes, and incorrect behavior

Example Prompts:

"Write a multithreaded TCP server in C with one thread per client."

"How to use sendmsg() with iovec for scatter-gather I/O in Linux?"

"Check whether this zero-copy socket code using MSG_ZEROCOPY is correct."

Validation: All AI-generated code was manually reviewed, modified, compiled, executed, and tested by me. The final code structure, logic corrections, and optimizations were performed by me after experimentation.

7.1.2 Experimental Design and Parameter Selection

AI tools were used to assist in:

- Selecting reasonable message sizes and thread counts
- Designing experiments that satisfy assignment requirements
- Understanding which metrics are meaningful for network I/O analysis

Example Prompts:

"What message sizes are reasonable for evaluating socket I/O performance?"

"How does thread count affect cache contention in multithreaded servers?"

Validation: Final experimental parameters were chosen by me and verified through repeated experimental runs on my system.

7.1.3 Running Experiments and Data Collection

AI tools were used to:

- Assist in writing automated bash scripts for experiment execution
- Suggest correct usage of `perf stat` and relevant counters
- Help parse profiling output into CSV format

Example Prompts:

"How to automate perf stat collection in a bash script?"

"Which perf counters are relevant for cache miss analysis?"

Validation: All experiments were executed by me on my local machine. All CSV files contain data obtained from actual runs and were not generated or fabricated by AI.

7.1.4 Plot Generation and Visualization

AI tools were used to:

- Generate Python scripts for plotting CSV data
- Suggest appropriate plot layouts, labels, and scales

Example Prompt:

"Generate matplotlib code to plot cache misses vs message size for multiple thread counts."

Validation: Plots were generated from real CSV data produced by my experiments. Axis values, scales, and trends correspond directly to measured results.

7.1.5 Data Interpretation and Report Writing

AI tools were used to:

- Help interpret observed performance trends
- Assist in explaining unexpected results
- Improve clarity, structure, and LaTeX formatting of the report and Readme

Validation: All explanations were verified against experimental data and OS-level behavior. Final wording and conclusions reflect my understanding and judgment.

7.2 Summary and Responsibility Statement

AI tools were used extensively across implementation, experimentation, visualization, and reporting. However:

- All code was compiled, executed, and debugged by me
- All experiments were run on LHC system
- All results correspond to real measurements
- All final interpretations and conclusions are my own

8 Conclusion

This assignment demonstrates that data movement is a dominant cost in network I/O. While zero-copy communication can significantly reduce CPU and cache overheads, its benefits are workload-dependent. One-copy communication provides a practical middle ground, reducing unnecessary copies without the complexity of full zero-copy mechanisms.

Understanding these trade-offs is essential for designing efficient, high-performance networked systems.

9 GitHub Repository

Repository URL:

https://github.com/ImPiyushK/GRS_PA02

All source files, scripts, data, and plots are available in the repository following the MT25076_Part_X naming convention.